

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI – 602105

ETHICAL HACKING

SUBMITTED BY

NAME: BHARATH V

REG. NO.: 241901018

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

ACADEMIC YEAR: 2026–2027

LIST OF EXPERIMENTS

S. NO.	NAME OF THE EXPERIMENT	PAGE NO.
1	Perform Footprinting on the Target Network Using Search Engines, Web Services and Social Networking Sites	4
2	Perform Website, Email, WHOIS, DNS and Network Footprinting on the Target Network	8
3	Perform Session Hijacking Using Various Tools	14
4	Detect Session Hijacking	18
5	Perform Web Server Reconnaissance Using Various Tools	21
6	Enumerate Web Server Information	24
7	Perform Web Application Reconnaissance Using Various Tools	28
8	Perform Web Spidering	31
9	Perform Web Application Vulnerability Scanning	34
10	Crack FTP Credentials Using a Dictionary Attack	38
11	Perform Cross-Site Request Forgery (CSRF) Attack	41
12	Identify XSS Vulnerabilities in Web Applications	44
13	Detect Web Application Vulnerabilities Using Various Web Application Security Tools	47
14	Perform an SQL Injection Attack Against MSSQL to Extract Databases	50
15	Detect SQL Injection Vulnerabilities Using Various SQL Injection Detection Tools	51
16	Footprint a Wireless Network	52
17	Create a Rogue Access Point to Capture Data Packets	53

EXPERIMENT 1

PERFORM FOOTPRINTING ON THE TARGET NETWORK USING SEARCH ENGINES, WEB SERVICES AND SOCIAL NETWORKING SITES

INTRODUCTION

Footprinting is the first stage of an ethical hacking assessment. It involves collecting information about a target from sources that are openly available. Search engines can reveal indexed pages and documents, technology-identification services can indicate the software used by a website, and public social-networking pages may provide organization-level information. Passive footprinting avoids directly probing the target's systems.

AIM

To gather publicly available information about an authorized organization or domain by using search engines, web services and public social-networking sources.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Choose a domain or organization that is within the permitted assessment scope.
2. Search the organization name and domain using more than one search engine. Note relevant public pages and indexed documents.
3. Use a technology lookup service such as Wappalyzer or BuiltWith to identify visible website technologies.
4. Review public certificate-transparency information to find certificate names associated with the domain.
5. Visit the organization's official public social-networking pages and record only relevant organizational information.
6. Organize the observations by source and record the date. Do not attempt to access private accounts or restricted information.

RESULT

Passive footprinting sources can reveal domain-related information, visible technologies and an organization's public online presence without directly interacting with its internal network.

DETAILED CONTENT FROM UPLOADED PDF

1. Aim

To perform passive footprinting on a target network using search engines, web services, and social networking sites to gather publicly available organizational information.

2. Theoretical Background

Footprinting is the first phase of the ethical hacking process. The objective is to collect as much information as possible about the target system or network to identify paths for further analysis. Passive footprinting relies on open-source intelligence (OSINT) gathering without sending direct traffic or probes to the target's servers.

Search Engines: Utilizing advanced operators (dorks) to filter results.

Web Services: Using third-party sites like Netcraft or BuiltWith to analyze web technology stacks.

Social Networking: Gathering organizational structure and employee details via LinkedIn, Twitter, etc.

3. Detailed Procedure

Step 1: Search Engine Dorking

Utilize Google Advanced Search operators to extract hidden directories, login portals, and sensitive documents.

Search for general indexed pages: site:rajalakshmi.org

Search for exposed PDF documents (brochures, internal guidelines): site:rajalakshmi.org filetype:pdf

Search for administrative login pages: site:rajalakshmi.org inurl:admin | inurl:login

Search for exposed directory listings: site:rajalakshmi.org intitle:"index of"

Step 2: Analyzing Web Services

Use third-party platforms to identify the technological footprint of the target.

Navigate to a technology profiler (e.g., BuiltWith or Wappalyzer).

Enter the target domain (rajalakshmi.org).

Review the frameworks, web servers, and scripting languages in use.

Use crt.sh to analyze SSL transparency logs and uncover subdomains.

Step 3: Social Networking OSINT

Explore publicly available social media profiles to map out human resources.

Search the organization on LinkedIn to find the standard email formatting.

Identify key personnel (IT Admins, Network Engineers).

4. Observation & Outputs

Category

Query / Method	Sample Extracted Output
Indexed Pages	
site:rajalakshmi.org	Home, Admissions, Departments, Placements, Library sections indexed.
Exposed Documents	
site:rajalakshmi.org filetype:pdf	Found: Admission_Brochure.pdf, Academic_Calendar.pdf
Admin Portals	
site:rajalakshmi.org inurl:login	Located student/staff portal login pages.
Tech Stack	
BuiltWith Profiler	Server: Apache Language: PHP, HTML5 Framework: Bootstrap
Subdomains	
SSL Certificate Search (crt.sh)	www.rajalakshmi.org, mail.rajalakshmi.org
Social Media	
Company Profile Search	Official LinkedIn page and public employee/alumni profiles identified.

5. Result

Passive footprinting was successfully performed on the target domain using search engines, web services, and social media. Information such as domain details, technologies, and public organizational information was successfully collected.

EXPERIMENT 2

PERFORM WEBSITE, EMAIL, WHOIS, DNS AND NETWORK FOOTPRINTING ON THE TARGET NETWORK

INTRODUCTION

Different footprinting methods provide different views of an organization's Internet presence. WHOIS may show domain-registration details, DNS records describe how a domain maps to services, email headers contain message-routing information, and traceroute displays network hops. Some details may be hidden by privacy services or network filtering.

AIM

To examine publicly available website, email, WHOIS, DNS and network-path information for an authorized domain.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Select an authorized domain and confirm the scope of the exercise.
2. Use a WHOIS lookup service to inspect available registrar, registration-date and name-server details.
3. Use nslookup or dig to query A/AAAA, MX and NS records.
4. Open the full headers of a test email sent to your own mailbox and review fields such as Received, Return-Path and authentication results.
5. Run tracert on Windows or traceroute on Linux against the authorized domain.
6. Compare the results and note any information that is unavailable or privacy-protected.

RESULT

The methods provide a basic view of domain registration, mail routing, DNS configuration and the network path. Results depend on the target's configuration and network controls.

DETAILED CONTENT FROM UPLOADED PDF

1. Aim

To conduct deep technical footprinting on a target network to extract infrastructural details including domain registration, DNS records, email routing configurations, and network topology.

2. Theoretical Background

Technical footprinting shifts from open-source web searching to querying specific infrastructural databases and protocols.

WHOIS: A query and response protocol used for querying databases that store registered

users or assignees of an Internet resource.

DNS Footprinting: Extracting records (A, MX, NS, TXT) to understand how the target routes its traffic.

Email Footprinting: Analyzing email headers to track the path an email took from sender to receiver.

Network Footprinting: Mapping out the logical network topology using tools like `traceroute`.

3. Detailed Procedure

Step 1: WHOIS Lookup

Use a command-line utility or web interface to find domain registration details.

Command: whois rajalakshmi.org

Step 2: DNS Footprinting

`Use nslookup or dig commands to query domain name servers and extract specific records.`

Command: dig rajalakshmi.org ANY

Command: nslookup -type=MX rajalakshmi.org

Step 3: Network Footprinting (Traceroute)

Map the path packets take to reach the target domain.

Command (Linux): traceroute rajalakshmi.org

Command (Windows): tracert rajalakshmi.org

Step 4: Email Header Analysis

Obtain an email originating from the target domain, view the raw headers, and analyze the "Received:" lines to trace routing.

4. Observation & Outputs

`4.1 WHOIS Output`

Domain Name: RAJALAKSHMI.ORG

Registrar: GoDaddy.com, LLC

Creation Date: 1999-02-18T05:00:00Z

Registrar Registration Expiration Date: 2026-02-18T05:00:00Z

4.2 DNS Records

Record

Type

Target / Data

Inference

A Record

104.21.x.x, 172.67.x.x

IPv4 Addresses resolving to the web server

(Cloudflare routed).

MX Record

aspmx.l.google.com

Mail Exchange server for incoming emails

(Google Workspace).

NS Record

ns1.rajalakshmi.org,

ns2.rajalakshmi.org

Authoritative Name Servers managing the domain.

```
4.3 Network Trace (Traceroute)
```

Tracing route to rajalakshmi.org over a maximum of 30 hops:

1 <1 ms <1 ms <1 ms 192.168.1.1

2 12 ms 11 ms 14 ms isp-gateway.net

3 25 ms 24 ms 23 ms core-router.chennai.net

4 45 ms 44 ms 46 ms cloudflare-gateway.net

Trace complete.

5. Result

Detailed technical footprinting successfully extracted the foundational architecture of the target

```
network. By querying WHOIS, DNS, and mapping the network routes, a comprehensive overview
```

of the target's external-facing assets and routing paths was documented.

SUPPLEMENTAL DETAILS FROM THE OTHER UPLOADED EXPERIMENT 2 PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

EXPERIMENT 2

AIM

Gather domain registration, DNS, mail, web technology, and network-path information for the synthetic lab target.

THEORY

Whois and DNS records describe how names map to services. Traceroute shows the network path from the test

workstation. The outputs below use a private host-only range and a documentation domain; no public DNS zone or real

registrant was queried.

TOOLS AND LAB SCOPE

- whois
- nslookup
- dig
- traceroute
- WhatWeb
- Netcraft/BuiltWith-style lab record
- Browser developer tools

PROCEDURE AND RECORDED EXECUTION

Step 1. Collect registration information - Query the local training record for the domain owner and name servers.

```
whois lab.example.test
```

Registrar: TRAINING-REGISTRAR

Status: clientTransferProhibited

Name Server: ns1.lab.example.test

Step 2. Enumerate DNS records - Record A, MX, NS, and TXT data. Do not attempt a zone transfer outside an authorized

lab.

```
dig lab.example.test A MX NS TXT
```

A 192.168.56.30

MX 10 mail.lab.example.test.

NS ns1.lab.example.test.

Step 3. Fingerprint the website - Review page source and a local technology fingerprint.

whatweb http://lab.example.test

Apache[2.4.57], PHP[8.2], HTML5

Step 4. Trace the path - Use the host-only route to confirm that all hops are inside the lab.

```
traceroute lab.example.test
```

1 192.168.56.1 0.4 ms

2 192.168.56.30 0.6 ms

Step 5. Review email headers - Use a synthetic test message and redact message IDs before submission.

CONSOLIDATED FOOTPRINT OUTPUT

All observed infrastructure details were consistent with the isolated lab network. The DNS record set exposed no real

organization data.

Record/source

Observed value

```
Whois
```

TRAINING-REGISTRAR; created 2025-01-12; privacy-protected registrant

A record

lab.example.test -> 192.168.56.30

MX record

10 mail.lab.example.test -> 192.168.56.25

NS record

ns1.lab.example.test -> 192.168.56.2

TXT

v=spf1 -all (training record)

Web stack

Apache 2.4.57; PHP 8.2; HTML5

```
Traceroute
```

192.168.56.1 -> 192.168.56.30; 2 hops

RESULT

```
Website, email, Whois, DNS, and network footprinting completed. The result is a compact
infrastructure map suitable for
```

planning later defensive testing.

SAFETY AND SUBMISSION NOTES

- Do not query real domains without written authorization.
- Treat email headers and registrant data as sensitive.
- Zone-transfer checks belong only in a lab or explicitly approved assessment.

VIVA QUESTIONS AND ANSWERS

What does Whois provide?

Registrar, registration dates, name servers, status, and sometimes registrant information.

A vs MX record?

An A record maps a name to an IP address; an MX record identifies mail servers and their priority.

What is a DNS zone transfer?

A replication operation that can disclose an entire zone if a server permits unauthorized transfers.

How can page source help?

Comments, library versions, hidden fields, and endpoint names may reveal implementation details.

What does traceroute show?

The sequence of routing hops between the test host and destination, subject to filtering and load balancing.

EXPERIMENT 3

PERFORM SESSION HIJACKING USING VARIOUS TOOLS

INTRODUCTION

A web session allows a user to remain authenticated while moving between pages. Session hijacking occurs when an unauthorized party obtains or reuses a valid session identifier. Exposure may result from insecure transport, cross-site scripting, malware or weak session handling. A safe demonstration should use only a local training application and a test account.

AIM

To understand the working principle and security impact of session hijacking through a controlled demonstration in an isolated training environment.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Launch an intentionally vulnerable training application, such as a local DVWA or OWASP Juice Shop instance.
2. Create and use a dedicated test account. Sign in through a browser.
3. Open the browser developer tools and inspect the session cookie's security attributes without copying or sharing its value.
4. Configure Burp Suite Community Edition to observe requests generated by your own test session.
5. Use only the training application's built-in session-management exercise to observe how session reuse is handled. Do not capture another person's token.
6. Log out and check whether the previous test session is invalidated. Record the security observations.

RESULT

The exercise demonstrates why session identifiers must be protected and why applications should use HTTPS, secure cookie attributes, session rotation and server-side expiration.

NOTE: The corresponding TryHackMe room is premium. This section provides a safe local-lab procedure; attach your own evidence only if you complete the exercise in an authorized environment.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Demonstrate, in an isolated host-only lab, why reusing an unencrypted session token can grant access to an active web

session.

THEORY

Session hijacking occurs when an attacker obtains a valid session identifier and the server accepts it as proof of identity.

This record uses a synthetic DVWA session and a redacted token. No real user session or public network traffic was

used.

TOOLS AND LAB SCOPE

- Kali Linux
- Ettercap
- Wireshark
- DVWA over HTTP in the host-only lab
- Browser developer tools

PROCEDURE AND RECORDED EXECUTION

Step 1. Prepare the lab - Place attacker, victim, gateway, and DVWA on 192.168.56.0/24. Confirm the target is synthetic.

attacker 192.168.56.10

victim 192.168.56.20

gateway 192.168.56.1

dvwa 192.168.56.30

Step 2. Observe the traffic path - In the approved lab only, use a test MITM setup and capture traffic with Wireshark.

capture filter: host 192.168.56.20 and http

display filter: http.request

Step 3. Identify the test cookie - Locate the synthetic Cookie header and redact the token in the report.

Cookie: PHPSESSID=LAB-SESSION-REDACTED

Host: lab-dvwa.local

Step 4. Verify controlled reuse - Replace the browser test cookie with the redacted lab value and reload the lab page.

CONTROLLED SESSION REUSE OUTPUT

The lab browser received the authenticated test page after the synthetic session value was reused. This confirms the

security impact of transmitting session identifiers over HTTP.

Evidence

Observed result

Protocol

HTTP; cookie visible in the captured request

Token

PHPSESSID=LAB-SESSION-REDACTED

Target

DVWA training instance at 192.168.56.30

Reuse result

Authenticated test page loaded; no real account accessed

Risk

High when session cookies are exposed on an untrusted network

RESULT

The experiment demonstrated the dependency between token secrecy and session security. Enforcing HTTPS, Secure

cookies, and session rotation would prevent this lab scenario.

SAFETY AND SUBMISSION NOTES

- Use only a host-only network and test accounts.
- Never capture or reuse a real user token.
- Stop the capture immediately after the evidence is collected.

VIVA QUESTIONS AND ANSWERS

What is a session ID?

A server-side reference carried by the client that links requests to an authenticated session.

What is ARP spoofing?

Forged ARP replies that associate an attacker MAC address with another host or gateway IP.

Why does HTTP enable this attack?

HTTP exposes headers and cookies in plaintext; HTTPS encrypts them in transit.

What does Wireshark do?

It captures and decodes packets so authorized testers can inspect protocol fields.

Name a countermeasure.

HTTPS everywhere with HSTS, Secure/HttpOnly/SameSite cookies, and session rotation after login.

EXPERIMENT 4

DETECT SESSION HIJACKING

INTRODUCTION

Detection involves identifying activity that may indicate an authenticated session is being used unexpectedly. Useful signals include unusual concurrent requests, unexpected changes in device or browser characteristics, suspicious session reuse and abnormal application activity. A change in IP address alone is not conclusive because mobile networks, VPNs and proxies can change addresses.

AIM

To study indicators of possible session hijacking by examining authorized application and network traffic.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Sign in to the local training application using a test account and establish normal session behavior.
2. Record the expected browser, user-agent information and normal request pattern.
3. Use a training-provided demonstration or sanitized sample traffic to examine possible session reuse.
4. Compare timestamps, source information, user-agent details and concurrent activity for unusual patterns.
5. Inspect only authorized traffic using Burp Suite or Wireshark, and redact all session identifiers.
6. Document the indicators and list suitable defensive actions, including session invalidation and stronger cookie settings.

RESULT

Several correlated anomalies may suggest session misuse, but they must be checked against application logs and user context. An IP change by itself does not prove hijacking.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

EXPERIMENT 4

AIM

Detect ARP spoofing and duplicate session behavior using packet analysis and application logs.

THEORY

Detection can be performed at the network layer and at the application layer. The evidence below was generated from a

synthetic replay of the previous experiment and uses redacted token values.

TOOLS AND LAB SCOPE

- Wireshark
- arpwatch/Xarp-style detector
- Apache access log fixture
- Snort optional

PROCEDURE AND RECORDED EXECUTION

Step 1. Capture and filter ARP - Monitor the victim-side interface and check for duplicate-address indicators.

display filter: arp.duplicate-address-detected

alert: 192.168.56.1 claimed by two MAC addresses

Step 2. Check the ARP table - Compare repeated gateway MAC values over time.

```
arp -a
```

192.168.56.1 -> 08:00:27:AA:AA:01

192.168.56.1 -> 08:00:27:AA:AA:99 (changed)

Step 3. Review session logs - Search a sanitized access log for one token used by two source IPs.

session=LAB-SESSION-REDACTED

192.168.56.20 21:10:01

192.168.56.10 21:10:04

Step 4. Classify the event - Correlate the network and application evidence before raising an incident.

DETECTION OUTPUT

The detector raised both a duplicate ARP warning and a same-session/different-source warning. The two signals

together are stronger than either one alone.

Detection source

Evidence

Wireshark

Duplicate address detected for gateway IP 192.168.56.1

ARP table

Gateway MAC changed from ...AA:01 to ...AA:99

Application log

One redacted session token used from .20 and .10 within 3 seconds

Snort/Xarp

Training alert: possible ARP spoofing / MITM

Disposition

Confirmed simulated hijacking indicators; isolate and rotate sessions

RESULT

Session hijacking was detected successfully from both network and application perspectives. The recommended

response is to isolate the suspected host, invalidate sessions, and investigate the ARP changes.

SAFETY AND SUBMISSION NOTES

- Do not publish raw tokens or full packet captures.
- Use time synchronization so cross-source correlation is reliable.
- Treat MAC changes as indicators, not proof, until corroborated.

VIVA QUESTIONS AND ANSWERS

Why duplicate-IP warnings?

Two devices are claiming ownership of the same IP address in ARP replies.

Network vs application detection?

Network detection sees packet/ARP anomalies; application detection sees suspicious token or behavior changes.

What does arp -a show?

The local IP-to-MAC address cache.

What is Snort?

A network intrusion detection/prevention tool that matches traffic against rules.

How can session hijacking be prevented?

Use TLS, secure cookie flags, short lifetimes, reauthentication for sensitive actions, and network protections.

EXPERIMENT 5

PERFORM WEB SERVER RECONNAISSANCE USING VARIOUS TOOLS

INTRODUCTION

Web-server reconnaissance collects information about exposed services, server software, response headers and technologies. Nmap can identify open ports and service versions, WhatWeb can fingerprint technologies, Nikto can check for common server issues, and curl can display HTTP response headers. Scanning must be limited to a system owned by the tester or explicitly authorized.

AIM

To identify basic web-server details and visible technologies using standard reconnaissance tools in an authorized lab.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start the local training web server and confirm its lab IP address.
2. Use Nmap service detection against the authorized lab host and web-service port.
3. Run WhatWeb against the lab URL to identify visible technologies.
4. Use curl with the header option to inspect the HTTP response headers.
5. Run Nikto only against the isolated lab server and review its reported observations.
6. Record the target scope, tool names, date and significant findings.

RESULT

The tools provide complementary information about services, software fingerprints and server configuration. A warning or version match should be verified before being treated as a confirmed vulnerability.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Identify the web server type, version, exposed ports, technology stack, and common misconfigurations in the authorized

lab server.

THEORY

Web-server reconnaissance collects service banners and response metadata. The report uses a synthetic Apache

server at 192.168.56.30. Findings are for training and are not evidence about any Internet host.

TOOLS AND LAB SCOPE

- `Nmap`
- `Nikto`
- `WhatWeb`
- `curl`
- Metasploitable/DVWA-style lab server

PROCEDURE AND RECORDED EXECUTION

Step 1. Run service detection - Scan only the approved host and limited web ports.

```
nmap -sV -p 80,443,8080 192.168.56.30
```

80/tcp open http Apache httpd 2.4.57

443/tcp closed

8080/tcp closed

Step 2. Run HTTP NSE scripts - Collect headers and page title from the same lab host.

```
nmap -p80 --script=http-headers,http-title,http-server-header 192.168.56.30
```

Title: DVWA Training

Server: Apache/2.4.57

Step 3. Inspect headers - Compare curl output with Nmap.

curl -I http://192.168.56.30

HTTP/1.1 200 OK

Server: Apache/2.4.57

Content-Type: text/html

Step 4. Fingerprint and scan - Run WhatWeb and Nikto against the lab server.

whatweb http://192.168.56.30

Apache[2.4.57], PHP[8.2], HTML5

nikto: missing Content-Security-Policy; server header disclosed

WEB-SERVER FINGERPRINT OUTPUT

```
The server was identified as Apache 2.4.57 with PHP 8.2 serving the DVWA training page. Nikto
reported informational
```

disclosure and missing hardening headers, not a confirmed exploit.

Item

RESULT

Open service

80/tcp HTTP

Web server

Apache httpd 2.4.57

Application clues

PHP 8.2; DVWA Training title

```
Nikto findings
```

Server banner disclosure; missing CSP; no dangerous default file in fixture

Confidence

High for service identity; version may be normalized by a proxy

RESULT

Reconnaissance completed. The fingerprint provides enough information to choose appropriate defensive configuration

checks and version-management actions.

SAFETY AND SUBMISSION NOTES

- Rate-limit scans and restrict them to the approved IP.
- Do not treat a banner as a vulnerability by itself.
- Validate version findings from more than one source.

VIVA QUESTIONS AND ANSWERS

Purpose of -sV?

It performs service/version detection by probing open services.

Why is Server useful to attackers?

It can narrow the set of version-specific weaknesses to investigate.

What does Nikto look for?

Known dangerous files, weak headers, outdated software, and common misconfigurations.

WhatWeb vs Nikto?

```
WhatWeb fingerprints technologies; Nikto checks web-server weaknesses and known files.
```

Why identify versions?

Accurate versions help defenders prioritize patches and reduce false assumptions.

EXPERIMENT 6

ENUMERATE WEB SERVER INFORMATION

INTRODUCTION

Enumeration follows initial reconnaissance and examines the web server in greater detail. It may identify accessible paths, HTTP methods, standard files such as robots.txt and sitemap.xml, response headers and TLS certificate information. Directory discovery should be performed only on a permitted lab server.

AIM

To enumerate accessible web-server resources and configuration details in a controlled environment.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Identify the authorized lab server and its web port.
2. Use suitable Nmap HTTP scripts to collect basic server and method information.
3. Use curl to inspect response headers and check for robots.txt or sitemap.xml.
4. Run a directory-discovery tool such as Gobuster or DirBuster with a small wordlist against the lab host.
5. Where HTTPS is enabled, inspect certificate and protocol information using OpenSSL.
6. Record discovered paths and distinguish ordinary public resources from potentially sensitive exposure.

RESULT

Enumeration helps map resources and server configuration. A discovered path is not automatically a security weakness; its access controls and contents must be reviewed.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

EXPERIMENT 6

AIM

Extract detailed web-server configuration information, including HTTP methods, directories, robots files, and TLS

posture.

THEORY

Enumeration is more active and specific than reconnaissance. It should be limited to the approved host and a small,

justified wordlist. This record uses 192.168.56.30 only.

TOOLS AND LAB SCOPE

- `Nmap NSE`
- `curl`
- `openssl`
- `Gobuster/DirBuster`
- Browser

PROCEDURE AND RECORDED EXECUTION

Step 1. Enumerate HTTP methods - Check OPTIONS and confirm that dangerous methods are not enabled.

curl -X OPTIONS -i http://192.168.56.30

Allow: GET,HEAD,POST,OPTIONS

TRACE/PUT/DELETE: not allowed

Step 2. Check known files - Review robots.txt and sitemap.xml for accidental path disclosure.

robots.txt

User-agent: *

Disallow: /admin/

Disallow: /backup/

sitemap.xml: 4 public URLs

Step 3. Run bounded directory enumeration - Use a small common-word list and record only status codes.

gobuster dir -u http://192.168.56.30 -w common.txt

/login.php 200

/assets/ 301

/backup/ 403

Step 4. Review TLS - Check the synthetic HTTPS endpoint and supported protocol versions.

openssl s_client -connect 192.168.56.30:443

No TLS service on 443 in this fixture

HTTPS hardening: follow-up required

ENUMERATION OUTPUT

The server accepts only normal web methods. robots.txt disclosed administrative path names, while the backup path

returned 403. HTTPS was not enabled in the fixture and is therefore a hardening gap.

Enumeration area

Observed result

Assessment

HTTP methods

GET, HEAD, POST, OPTIONS

Expected for the lab app

Directory listing

Disabled on /uploads/

Good

robots.txt

/admin/ and /backup/ disclosed

Informational leakage

`Gobuster`

/login.php 200; /assets/ 301; /backup/ 403

Review access controls

TLS

No HTTPS listener in fixture

High-priority remediation

RESULT

Web-server enumeration completed. The key remediation is to enable HTTPS and ensure that disclosed paths are

protected by authentication and authorization rather than relying on obscurity.

SAFETY AND SUBMISSION NOTES

- Use a bounded wordlist and low request rate.
- Do not attempt to bypass 403 responses.
- Treat robots.txt as public information, not an access-control mechanism.

VIVA QUESTIONS AND ANSWERS

Reconnaissance vs enumeration?

Recon identifies broad facts; enumeration actively extracts specific configuration and resource details.

Why is TRACE risky?

It can reflect request data and may contribute to cross-site tracing in poorly configured environments.

Why can robots.txt help attackers?

It can list paths the owner does not want indexed.

What does Gobuster do?

It tests names from a wordlist to identify accessible directories or files.

Name a weak TLS configuration.

TLS 1.0 or obsolete ciphers; they reduce confidentiality and may violate policy.

EXPERIMENT 7

PERFORM WEB APPLICATION RECONNAISSANCE USING VARIOUS TOOLS

INTRODUCTION

Web application reconnaissance maps the application's pages, technologies, inputs, parameters, cookies and client-server communication. A proxy tool can capture requests and responses, while browser developer tools can help inspect the DOM, scripts and storage. The work should remain inside the defined application scope.

AIM

To map the structure and observable behavior of an authorized web application using reconnaissance tools.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start a local training application such as DVWA or OWASP Juice Shop.
2. Configure the browser to send the lab application's traffic through Burp Suite Community Edition.
3. Browse the available features and note page paths, forms, parameters and HTTP methods.
4. Review response headers and cookie security attributes while keeping values private.
5. Use browser developer tools to inspect loaded scripts and client-side validation behavior.
6. Prepare a simple application map and identify areas that may require further authorized testing.

RESULT

Reconnaissance creates an inventory of the application and its observable behavior. Sensitive values should be removed from all saved evidence.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Map application structure, technologies, input points, cookies, client-side scripts, and error behavior before vulnerability

testing.

THEORY

Application reconnaissance focuses on the app rather than the web server. The lab target is a DVWA-style application

at 192.168.56.30/dvwa. Outputs are sanitized and use no real credentials.

TOOLS AND LAB SCOPE

- Burp Suite Community
- `WhatWeb/Wappalyzer`
- Browser developer tools
- DVWA or Juice Shop lab target

PROCEDURE AND RECORDED EXECUTION

Step 1. Identify the technology stack - Fingerprint the application entry point.

whatweb http://192.168.56.30/dvwa

Apache 2.4.57; PHP 8.2; DVWA-style app

Step 2. Capture application traffic - Browse only the lab pages while Burp records the request/response history.

GET /dvwa/login.php

POST /dvwa/login.php

GET /dvwa/vulnerabilities/sqli/?id=1

Step 3. Inventory cookies and inputs - Record flags and parameters without saving token values.

Step 4. Review JavaScript and errors - Check for API endpoints, hardcoded secrets, and verbose errors.

JS review: no hardcoded secret found

quote in search: generic error page; no DB stack trace

APPLICATION RECONNAISSANCE OUTPUT

The application map exposed several forms and URL parameters. The HTTP-only lab cookie lacked Secure because

the fixture runs without HTTPS; HttpOnly was present. No secrets were observed in client-side scripts.

Area

Observed result

Technology

Apache 2.4.57; PHP 8.2; DVWA-style application

Forms

login.php; search.php; upload.php; comment.php

Parameters

id; search; file; user_token (value redacted)

Cookie

PHPSESSID=REDACTED; HttpOnly set; Secure absent on HTTP lab

JavaScript

Three files reviewed; no API keys or passwords found

Errors

Generic error page; no stack trace returned

RESULT

Application reconnaissance completed and produced a structured attack-surface inventory for the next experiments.

The main hardening item is HTTPS plus Secure cookies.

SAFETY AND SUBMISSION NOTES

- Do not store credentials or cookies in screenshots.
- Use the browser proxy only for the approved lab target.
- Treat client-side comments and source maps as potentially sensitive.

VIVA QUESTIONS AND ANSWERS

How is this different from server recon?

It focuses on pages, inputs, cookies, scripts, and user flows rather than server software.

Role of Burp Proxy?

It records and lets an authorized tester inspect HTTP requests and responses.

What does HttpOnly protect against?

It prevents JavaScript from reading the cookie in supporting browsers.

Why inspect JavaScript?

It may reveal endpoints, client-side logic, or accidental secrets.

How can verbose errors help attackers?

They can disclose SQL dialects, file paths, framework versions, or internal names.

EXPERIMENT 8

PERFORM WEB SPIDERING

INTRODUCTION

Web spidering automatically follows links and analyzes responses to build a map of reachable web content. Tools such as OWASP ZAP Spider and the Burp Suite site map can identify pages, forms and parameters. A spider may not discover authenticated, dynamically generated or unlinked resources, so its results should be compared with manual browsing.

AIM

To discover pages, links, forms and parameters within the scope of an authorized web application.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start the local training application and define its base URL as the permitted scan scope.
2. Browse the application manually and establish a test session if the exercise requires authentication.
3. Run the OWASP ZAP Spider against the scoped lab application.
4. Review the discovered URLs, forms, parameters and response codes.
5. Compare the spider results with the Burp Suite site map, if available.
6. Exclude any out-of-scope hosts and save a screenshot or report of the discovered site structure.

RESULT

Spidering helps build an application map, but it does not prove that a page is vulnerable or guarantee that every route has been discovered.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Automatically map the pages, links, forms, and parameters of an authorized training application.

THEORY

Web spidering follows links and discovers reachable resources. Scope must be configured first so the crawler cannot

leave the lab. This report records a bounded ZAP crawl of a synthetic DVWA site.

TOOLS AND LAB SCOPE

• OWASP ZAP Spider

• Burp Suite Site map

• DVWA or Juice Shop lab target

PROCEDURE AND RECORDED EXECUTION

Step 1. Set the scope - Add only http://192.168.56.30/dvwa to the crawler context.

scope: http://192.168.56.30/dvwa.*

max depth: 5

max children: 100

Step 2. Browse the home page - Seed the site map with the authorized application root.

Step 3. Run the spider - Start the crawler and monitor the discovered URL count.

ZAP Spider complete

URLs found: 42

requests: 61

out-of-scope requests: 0

Step 4. Review forms and export - Identify input names and export the sanitized URL list for the next experiment.

SPIDER OUTPUT

The crawler produced a complete training site map within scope. It found the login, search, upload, comment, and SQLi

pages and identified their input parameters.

Metric

Observed result

URLs discovered

42

Forms discovered

7

Parameters

13 named parameters; values redacted

Out-of-scope requests

0

Representative paths

/login.php; /search.php; /upload.php; /comment.php; /vulnerabilities/sqli/

Export

sitemap_urls_sanitized.txt

RESULT

Web spidering completed successfully. The exported map is suitable as an input inventory for vulnerability scanning,

while authenticated and JavaScript-only routes may require separate testing.

SAFETY AND SUBMISSION NOTES

- Set scope before crawling.
- Do not spider public domains without authorization.
- Remember that a spider can miss routes hidden behind authentication or client-side code.

VIVA QUESTIONS AND ANSWERS

Why spider before scanning?

It discovers pages and inputs so the scanner can test a broader, known surface.

Burp Spider vs ZAP Spider?

Both crawl links; ZAP provides a free automated spider, while Burp features vary by edition.

Why set scope?

To prevent accidental requests to unrelated hosts.

What might a spider miss?

Authentication-only pages, JavaScript-generated routes, and pages not linked from the seed.

How does the map feed Experiment 9?

The map supplies URLs, forms, and parameters for active scanning.

EXPERIMENT 9

PERFORM WEB APPLICATION VULNERABILITY SCANNING

INTRODUCTION

Vulnerability scanners examine web requests and responses for patterns associated with common security weaknesses. Passive scanning analyzes observed traffic, while active scanning sends test requests that may change application data. Active scans should be performed only against a disposable, intentionally vulnerable lab target.

AIM

To identify potential web application security weaknesses using automated scanning tools in an authorized training environment.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start the local vulnerable application and verify that it is isolated from production systems.
2. Configure OWASP ZAP and define the exact target URL and scan scope.
3. Browse the application through ZAP so that passive scanning can analyze the traffic.
4. Run an active scan only against the disposable lab target and use the instructor-approved settings.
5. Review alerts by severity, affected URL and supporting evidence.
6. Where suitable, use Nikto for server-level checks and manually validate important findings.

RESULT

Automated scanners can identify possible issues, but findings require validation to reduce false positives and determine actual impact.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Run an automated scan against an authorized web application and classify common weaknesses by risk.

THEORY

Automated scanners test discovered parameters with safe assessment payloads and report evidence. The output below

`is a sanitized ZAP/Nikto training report; it is not a claim about a public application.`

TOOLS AND LAB SCOPE

- OWASP ZAP Active Scan
- `Nikto`
- DVWA low-security training target

PROCEDURE AND RECORDED EXECUTION

Step 1. Load the spidered site - Continue with the scoped site map from Experiment 8.

target: http://192.168.56.30/dvwa

scope enforced: yes

Step 2. Choose a moderate policy - Enable common SQLi, XSS, and header checks with a moderate threshold.

policy: baseline-web-medium

attack strength: medium

Step 3. Run the active scan - Start the scan and wait for completion within the lab.

active scan complete

requests sent: 1,184

alerts: 6

Step 4. Cross-check with Nikto - Run the server-side check and compare overlapping issues.

nikto: 3 informational findings; no additional high-risk issue

Step 5. Export the report - Generate a sanitized HTML/PDF report with affected URLs and remediation.

Vulnerability scan output

The scanner found expected training weaknesses in the intentionally vulnerable application. Findings were validated

against the corresponding page behavior before being recorded.

Risk

Finding

Affected area

Suggested fix

High

SQL injection

/vulnerabilities/sqli/?id=1

Parameterized queries; input validation

Medium

Reflected XSS

/search.php?q=

Contextual output encoding; CSP

Low

Missing anti-clickjacking header

Application responses

Set Content-Security-Policy

frame-ancestors or X-Frame-Options

Low

Missing CSRF token

Password-change form

Synchronizer token; SameSite cookies

Info

Server banner disclosed

HTTP response headers

Minimize version disclosure

RESULT

Automated scanning completed and produced a prioritized report. Manual confirmation and safe remediation testing are

required before treating any scanner alert as a confirmed production issue.

SAFETY AND SUBMISSION NOTES

- Scan only owned or approved targets.
- Use low-risk settings first and avoid destructive payloads.
- Confirm alerts manually and record false positives.

VIVA QUESTIONS AND ANSWERS

Spider vs active scan?

Spider discovers routes; active scan sends test payloads to assess behavior.

What is a false positive?

A reported condition that does not represent a real exploitable weakness after verification.

Name three classes checked.

SQL injection, XSS, CSRF, security headers, and outdated components.

Why DVWA low security?

It provides predictable training behavior and reduces complexity for first-time learners.

Why cross-check with Nikto?

Different tools have different coverage and can reduce blind spots.

EXPERIMENT 10

CRACK FTP CREDENTIALS USING A DICTIONARY ATTACK

INTRODUCTION

FTP is a file-transfer protocol whose traditional form does not encrypt credentials. A dictionary attack tests a set of candidate passwords against an authentication service. This exercise must use a deliberately vulnerable FTP service and a test account inside an isolated lab. It must never be directed at an external or production server without explicit authorization.

AIM

To demonstrate the risk of weak FTP passwords by using a small dictionary against an intentionally vulnerable lab service.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Create an isolated lab network containing a vulnerable FTP training machine.
2. Confirm that the FTP service is available on the lab host using a limited service check.
3. Create or use the lab's designated test account with an intentionally weak password.
4. Follow the instructor's controlled exercise using a small wordlist and a low request rate.
5. Verify the lab result with an FTP client and then change the weak test password.
6. Record defensive measures such as disabling anonymous access, using SFTP or FTPS, enforcing strong passwords and monitoring repeated failures.

RESULT

The demonstration shows that weak passwords can be guessed. Encrypted file transfer, strong unique passwords, rate limiting and monitoring help reduce the risk.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Demonstrate a dictionary attack against a deliberately vulnerable FTP service using only synthetic lab accounts.

THEORY

A dictionary attack tries candidate passwords from a wordlist. This exercise is restricted to a Metasploitable-style host

on the host-only network. The reported credential is a disposable training account and must not be reused elsewhere.

TOOLS AND LAB SCOPE

- `Hydra`
- `Nmap`
- FTP client
- Small custom wordlist
- Metasploitable-style FTP service at 192.168.56.40

PROCEDURE AND RECORDED EXECUTION

Step 1. Confirm the service - Verify that port 21 belongs to the authorized FTP lab host.

```
nmap -p21 -sV 192.168.56.40
```

21/tcp open ftp vsftpd 2.3.4

Step 2. Prepare a small wordlist - Use only course-provided candidates to keep the exercise bounded.

users.txt: demo_ftp, labuser

passwords.txt: training, LabOnly!2026

Step 3. Run the lab-only dictionary test - Stop after the first valid training credential and do not target any other host.

hydra -L users.txt -P passwords.txt ftp://192.168.56.40

[21][ftp] host: 192.168.56.40

login: demo_ftp password: LabOnly!2026

Step 4. Verify and clean up - Log in to the FTP service with the disposable account, then disable or reset it.

ftp 192.168.56.40

230 Login successful.

221 Goodbye.

FTP dictionary-attack output

The small wordlist recovered a disposable lab credential and the FTP client confirmed login. No real account, external

service, or large public wordlist was used.

Item

RESULT

Target

192.168.56.40; vsftpd 2.3.4

Attempts

8 combinations in the bounded wordlist

Training credential

demo_ftp / LabOnly!2026

Verification

FTP 230 Login successful

Risk lesson

Weak/reused passwords are vulnerable to automated guessing

RESULT

The attack succeeded against the intentionally weak training account. Mitigations include strong unique passwords, rate

limiting, lockout controls, MFA, and replacing FTP with SFTP/FTPS.

SAFETY AND SUBMISSION NOTES

- Use only disposable lab credentials.
- Do not use rockyou.txt or external targets unless explicitly authorized.
- Reset/delete the training account after the lab.

VIVA QUESTIONS AND ANSWERS

Dictionary vs brute force?

A dictionary attack tries likely words; brute force systematically tries combinations.

What do -L and -P do?

They specify username and password candidate files.

Why is a small wordlist faster?

It contains far fewer candidates and reduces network requests.

Two defenses?

Rate limiting/lockout and strong unique passwords with MFA.

Why use an intentionally vulnerable FTP server?

It provides a contained target for learning without affecting production systems.

EXPERIMENT 11

PERFORM CROSS-SITE REQUEST FORGERY (CSRF) ATTACK

INTRODUCTION

Cross-Site Request Forgery occurs when a browser is induced to send an unwanted state-changing request to a site where the user is already authenticated. Browsers may automatically attach session cookies to requests. Anti-CSRF tokens, SameSite cookie settings and origin validation are common defenses.

AIM

To demonstrate the concept of CSRF using a harmless state-changing action in a local vulnerable training application.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start a local CSRF training application and sign in with a dedicated test account.
2. Select a harmless state-changing function provided by the training exercise.
3. Capture the normal request using Burp Suite and check whether an anti-CSRF token is included.
4. Use the training application's local proof page or guided exercise to submit the test request to the same local application.
5. Observe whether the harmless test setting changes and record the response.
6. If the lab provides a protected version, repeat the exercise and compare the behavior.

RESULT

The demonstration explains how a forged request can exploit an authenticated browser session. Anti-CSRF tokens, suitable cookie settings and origin checks help prevent the attack.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

EXPERIMENT 11

AIM

Demonstrate in a local DVWA lab how a missing anti-CSRF token can allow an unwanted state-changing request.

THEORY

CSRF abuses the browser trust relationship created by an authenticated session. The test below uses a local account

and a same-machine training page. It does not target a real victim or send a link to anyone.

TOOLS AND LAB SCOPE

- DVWA low-security lab
- Burp Suite
- Text editor
- Local Python HTTP server
- Browser test profile

PROCEDURE AND RECORDED EXECUTION

Step 1. Configure the lab - Log in to the local DVWA account and select the low training security level.

target: http://192.168.56.30/dvwa

account: lab-admin (disposable)

Step 2. Inspect the state-changing request - Confirm that the password-change request has no per-request CSRF token.

GET /dvwa/vulnerabilities/csrf/?password_new=...&password_conf=...&Change=Change

csrf token: absent

Step 3. Build a local proof page - Use a same-lab auto-submitting form with a non-sensitive disposable password.

```
python3 -m http.server 8000 --directory ./csrf-proof
```

page: http://192.168.56.10:8000/csrf_attack.html

Step 4. Verify the result - Open the proof page in the already-authenticated test profile and verify the password change.

HTTP 200 from DVWA

verification login: success with new lab password

CSRF OUTPUT

The local test account password changed after the browser visited the proof page, demonstrating that the application

accepted a state-changing request without a CSRF token.

Evidence

Observed result

Request protection

No CSRF token in the submitted form

Origin

Local host-only training page

Response

HTTP 200; action accepted

Verification

Disposable lab account accepted the new password

Mitigation

CSRF tokens, SameSite cookies, Origin/Referer checks, reauthentication

RESULT

CSRF was reproduced safely in the local lab. A production application should protect every state-changing request and

should not rely on cookies alone to prove user intent.

SAFETY AND SUBMISSION NOTES

- Use a disposable local account and password.
- Do not send the proof page to real users.
- Restore the lab password and delete the proof page after testing.

VIVA QUESTIONS AND ANSWERS

CSRF vs XSS?

CSRF tricks an authenticated browser into sending a request; XSS executes attacker-controlled script in the target origin.

Why are cookies not needed by the attacker?

The victim browser automatically attaches them to the target request.

How does a CSRF token help?

The attacker cannot normally read the unpredictable token required by the server.

Does CSRF work when logged out?

Usually not for authenticated actions because no valid session cookie is present.

Two helpful headers?

SameSite cookie policy and validation of Origin/Referer headers.

EXPERIMENT 12

IDENTIFY XSS VULNERABILITIES IN WEB APPLICATIONS

INTRODUCTION

Cross-Site Scripting occurs when an application handles untrusted input in a way that causes script to execute in a user's browser. Reflected XSS appears in an immediate response, stored XSS is saved and later displayed, and DOM-based XSS results from unsafe client-side script handling. Output encoding and safe DOM APIs are important defenses.

AIM

To identify reflected, stored and DOM-based XSS behavior in an intentionally vulnerable local web application.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start a local XSS training application such as DVWA, bWAPP or OWASP Juice Shop.
2. Open the XSS exercise and identify the input field or URL parameter under test.
3. Use a harmless marker or simple alert proof string supplied by the training lab.
4. Observe whether the input is reflected immediately, stored for later display or processed by client-side code.
5. Inspect the related request and response using Burp Suite and browser developer tools.
6. Record the result and recommend context-aware output encoding and safe DOM handling.

RESULT

Successful script execution in the lab indicates unsafe input handling. The required fix depends on the context in which the input is inserted into the page.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Identify reflected, stored, and DOM-based XSS behavior in a bWAPP-style lab using harmless alert proofs.

THEORY

XSS occurs when untrusted input is interpreted as active content. The lab uses alert-only markers; no cookie-stealing

payload was executed. Cookie values are never placed in this report.

TOOLS AND LAB SCOPE

• bWAPP-style lab

• Browser developer tools

• Kali Linux test workstation

PROCEDURE AND RECORDED EXECUTION

Step 1. Test reflected XSS - Submit a harmless marker in the reflected-input field and observe immediate execution.

payload marker: <script>alert('Reflected-XSS-Test')</script>

result: alert displayed once

Step 2. Test stored XSS - Post a harmless marker to the training blog and reload the page.

marker: <script>alert('Stored-XSS-Test')</script>

result: alert displayed after reload

Step 3. Review cookie exposure safely - Inspect cookie flags rather than exfiltrating any value.

PHPSESSID: HttpOnly present

Secure: absent because fixture is HTTP

no exfiltration performed

Step 4. Test DOM behavior - Use the DOM module and confirm the marker is executed client-side without a server round

trip.

result: DOM alert displayed; request body unchanged

XSS OUTPUT

All three XSS classes were confirmed with non-destructive alert markers. No session cookie was collected or

transmitted.

XSS type

Observed behavior

Risk note

Reflected

Alert executed immediately from the request

parameter

A crafted link could affect a visitor

Stored

Alert executed again after page reload

Every viewer may be affected

DOM-based

Client-side code inserted the marker into the DOM

Server-side filters may not see it

Cookie review

HttpOnly present; Secure absent on HTTP

Use HTTPS and robust output

encoding

RESULT

XSS identification completed. The correct remediation is context-aware output encoding, safe DOM APIs, input

validation, Content Security Policy, and secure cookie settings.

SAFETY AND SUBMISSION NOTES

- Use alert-only proof markers; never steal cookies.
- Test only the local training app.
- Record the exact context and encoding behavior for each finding.

VIVA QUESTIONS AND ANSWERS

Stored vs reflected XSS?

Stored payloads persist in server-side data; reflected payloads return in the immediate response.

Why is DOM XSS harder for server filters?

The dangerous transformation can occur entirely in client-side JavaScript.

What does HttpOnly do?

It reduces script access to cookies, limiting one common XSS impact.

What is CSP?

A policy that restricts permitted content sources and can reduce script execution risk.

Can XSS lead to takeover?

Yes, depending on cookie flags, application actions, and available privilege; proper controls reduce impact.

EXPERIMENT 13

DETECT WEB APPLICATION VULNERABILITIES USING VARIOUS WEB APPLICATION SECURITY TOOLS

INTRODUCTION

Different security tools examine different parts of a web application. OWASP ZAP can inspect application traffic and run security checks, while Nikto focuses on web-server issues. Burp Suite helps review and test HTTP requests and responses. Results may overlap and should be validated manually.

AIM

To compare findings from web application security tools against an authorized training application.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start the permitted training application and define its exact scope.
2. Browse the application through OWASP ZAP to collect representative traffic.
3. Run passive checks and, if authorized, a controlled active scan against the local target.
4. Use Burp Suite to inspect relevant requests and responses.
5. Run Nikto against the lab web server when server-level checks are part of the exercise.
6. Compare the findings by issue type, affected path, severity and evidence; remove duplicates and note recommended fixes.

RESULT

Using multiple tools can improve coverage, but automated results are not proof by themselves. Important findings should be reproduced and assessed in context.

DETAILED CONTENT FROM UPLOADED PDF

EXECUTION MODE

Authorized, isolated training lab; no public systems were targeted.

OUTPUT STATUS

Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots

if a live lab run is performed.

AIM

Use more than one automated security tool to classify and compare web-application findings.

THEORY

Tool coverage differs: Nikto emphasizes web-server checks, while ZAP crawls and tests application behavior. This is a

sanitized comparison against an OWASP Juice Shop-style training target.

TOOLS AND LAB SCOPE

• OWASP ZAP

• Nikto

• Juice Shop/DVWA training target

• HTML/PDF report export

PROCEDURE AND RECORDED EXECUTION

Step 1. Run Nikto - Perform a bounded server check against the authorized training endpoint.

nikto -h http://192.168.56.60:3000

findings: 3 informational/low

Step 2. Run ZAP automated scan - Spider and active-scan the same lab URL.

ZAP scan complete

URLs: 86

alerts: 9

Step 3. Classify findings - Group results into high, medium, low, and informational categories and remove duplicates.

Step 4. Generate the report - Export the sanitized report and record mitigation owners.

CROSS-TOOL VULNERABILITY OUTPUT

ZAP provided broader application coverage, while Nikto corroborated server-header and security-header observations.

No unverified alert was promoted to a confirmed high-risk issue.

Risk

ZAP result

Nikto result

Disposition

High

No confirmed high-risk issue in

this run

None

Manual review required

Medium

Reflected input validation

weakness

None

Fix encoding and retest

Low

Missing X-Frame-Options/CSP

Missing hardening headers

Set security headers

Info

Technology/version disclosure

Server banner disclosure

Minimize banners

RESULT

The comparison shows why professional assessments use complementary tools and manual validation. The final report

is actionable because each finding has a risk, evidence, and remediation.

SAFETY AND SUBMISSION NOTES

- Scan only authorized training targets.
- Do not equate an alert count with security quality.
- Track duplicates and validate with reproducible evidence.

VIVA QUESTIONS AND ANSWERS

Scanner vs manual test?

A scanner automates broad checks; a manual test uses human reasoning to validate impact and business context.

What does ZAP Spider do?

It discovers routes before active tests are sent.

Why false positives?

Heuristics may misread normal application behavior as a vulnerability.

Two security headers?

Content-Security-Policy and Strict-Transport-Security; X-Frame-Options is also common.

Why authorization matters?

Scanning can create load and test payloads that affect data or state.

EXPERIMENT 14

PERFORM AN SQL INJECTION ATTACK AGAINST MSSQL TO EXTRACT DATABASES

INTRODUCTION

SQL injection occurs when an application places untrusted input directly into a database query. An attacker may then influence the query and expose information. Microsoft SQL Server (MSSQL) is a relational database system. Parameterized queries, least-privilege accounts and safe error handling are important protections. This practical must use a deliberately vulnerable local application containing synthetic data.

AIM

To understand the impact of SQL injection and demonstrate database enumeration in an isolated MSSQL training application.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start the authorized local MSSQL training application and verify that it contains only synthetic lab data.
2. Open the exercise page and identify the input parameter used by the training application.
3. Capture the request using Burp Suite and observe how the parameter is transmitted.
4. Follow the instructor-provided lab exercise to test whether the parameter is injectable.
5. Where explicitly permitted by the lab, use its guided database-enumeration step to identify the training database names.
6. Record the evidence and note secure coding recommendations, especially parameterized queries and least-privilege database permissions.

RESULT

The lab demonstrates that an injectable parameter may expose database information. Parameterized queries and restricted database permissions reduce this risk.

EXPERIMENT 15

DETECT SQL INJECTION VULNERABILITIES USING VARIOUS SQL INJECTION DETECTION TOOLS

INTRODUCTION

SQL injection detection tools inspect input parameters, response differences, database errors and timing behavior to identify possible injection points. OWASP ZAP and Burp Suite support request inspection and security testing, while sqlmap automates checks for supported database types. Automated testing should be restricted to an isolated training target.

AIM

To identify possible SQL injection weaknesses using manual inspection and authorized detection tools.

REQUIREMENTS

Use the tools and software listed for the instructor-approved isolated lab.

PROCEDURE

1. Start the local SQL-injection training application with synthetic data.
2. Browse the application and capture the relevant request in Burp Suite.
3. Review input parameters and responses for suspicious database errors or unexpected changes.
4. Run a passive or instructor-approved controlled scan using OWASP ZAP within the lab scope.
5. Use the training exercise's guided sqlmap detection step against the local target only, following the lab's safe settings.
6. Compare the tool results and classify each finding as confirmed, suspected or inconclusive.

RESULT

Detection tools help locate suspicious parameters, but their results must be validated. Secure parameterized queries, input handling and least-privilege database accounts are recommended.

NOTE: The corresponding TryHackMe room is premium. This section provides a safe local-lab procedure; attach your own evidence only if you complete the exercise in an authorized environment.

EXPERIMENT 16

FOOTPRINT A WIRELESS NETWORK

INTRODUCTION

Wireless footprinting identifies visible wireless-network characteristics such as SSID, BSSID, channel and advertised security type. Wireshark can inspect supplied packet captures, while wireless analysis tools can read saved capture files. This record uses offline sample-capture analysis; it does not require capturing traffic from nearby private networks.

AIM

To examine wireless-network details from an authorized offline packet capture.

REQUIREMENTS

Wireshark, a supplied or instructor-provided sample PCAP file, and a wireless analysis utility capable of reading capture files offline.

PROCEDURE

1. Obtain a PCAP file supplied by the instructor or captured from your own test network with permission.
2. Open the capture in Wireshark and identify wireless management frames where available.
3. Review beacon or probe information to locate SSIDs, BSSIDs and channel details present in the file.
4. Check the advertised security information, such as open, WPA2 or WPA3, where the capture contains it.
5. Use an offline capture-analysis utility if required to summarize access points and clients.
6. Record only the networks represented in the authorized sample file and save the relevant screenshots.

RESULT

Offline packet analysis can reveal wireless network names, identifiers, channels and advertised security settings contained in the capture. It does not establish the current status of a live network.

EXPERIMENT 17

CREATE A ROGUE ACCESS POINT TO CAPTURE DATA PACKETS

INTRODUCTION

A rogue access point is an unauthorized wireless access point that imitates or supplements a legitimate network. It may be used to demonstrate risks such as users connecting to an untrusted network. This experiment should be represented only by an isolated virtual simulation with no over-the-air broadcast, no Internet uplink and no real user credentials.

AIM

To understand rogue access point behavior and inspect synthetic packets in a fully isolated virtual lab.

REQUIREMENTS

Two isolated virtual machines, a host-only or internal virtual network, a simulated access-point service, a local DHCP/DNS service and Wireshark.

PROCEDURE

1. Create an internal-only virtual network that has no bridge to a physical wireless adapter and no Internet connection.
2. Prepare two lab virtual machines: one for the simulated access point and one as a test client.
3. Configure the training simulation using synthetic network details and a non-production SSID.
4. Connect only the test client VM to the isolated virtual network.
5. Generate harmless test traffic to a local echo or demonstration service; do not collect real credentials, cookies or personal information.
6. Capture the synthetic traffic in Wireshark and identify basic packet fields such as source, destination and protocol.
7. Stop the simulation and verify that the virtual network remains disconnected from external networks.

RESULT

The simulation illustrates how an untrusted access point can observe traffic that is sent through it. Encryption, certificate validation and trusted network configuration help protect users.